

Prieto Eats — Guía de estudio y defensa del proyecto

Proyecto Laravel con Breeze (Blade) + PostgreSQL

Aplicación de gestión de menús del departamento de Hostelería

Índice

- 1. [Arquitectura general \(MVC\)](#)
- 2. [Base de datos y relaciones](#)
- 3. [Modelos Eloquent](#)
- 4. [Sistema de rutas](#)
- 5. [Autenticación: login, registro y logout](#)
- 6. [Vistas Blade: layout y parciales](#)
- 7. [Home: ofertas con tabs y cards](#)
- 8. [Carrito de la compra con sesiones](#)
- 9. [Confirmar pedido \(transacción\)](#)
- 10. [Roles y middleware IsAdmin](#)
- 11. [CRUD de Productos \(Admin\)](#)
- 12. [Crear Ofertas con checkboxes \(Admin\)](#)
- 13. [Seeders y datos de prueba](#)
- 14. [Conceptos clave para defender el proyecto](#)

1. Arquitectura general (MVC)

Laravel sigue el patrón **MVC (Modelo – Vista – Controlador)**:

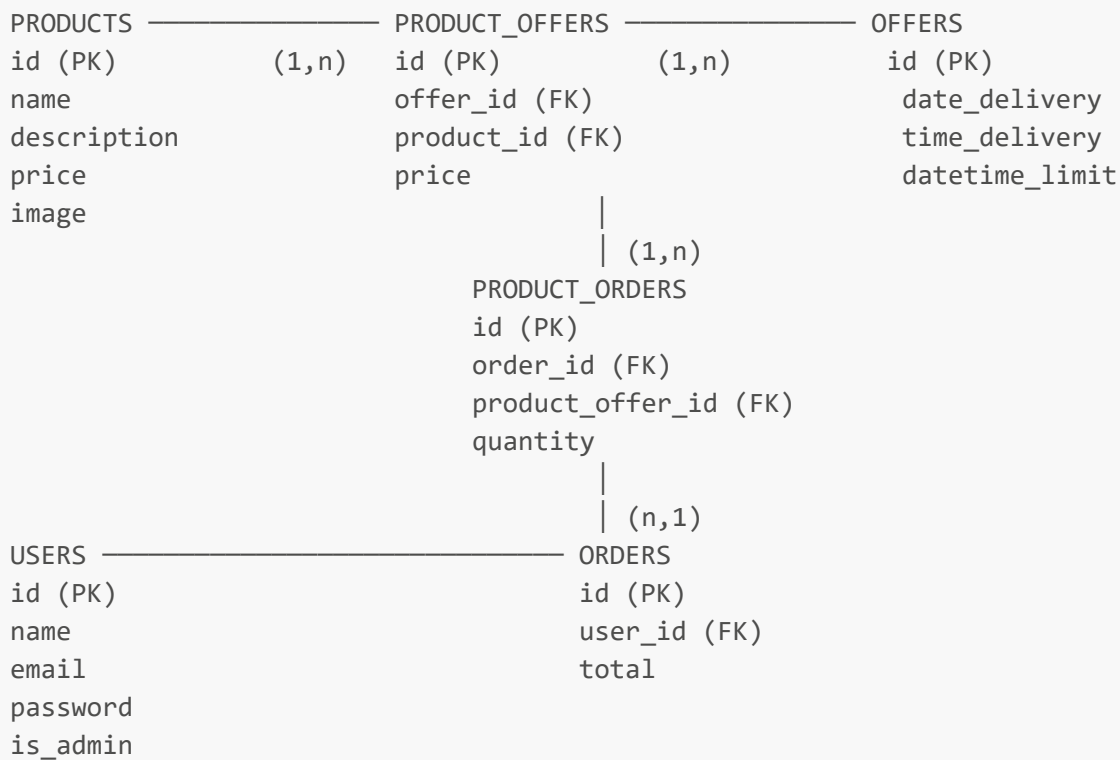


En este proyecto:

Capa	Dónde está	Qué hace
Modelo	app/Models/	Define tablas, campos rellenables y relaciones Eloquent
Vista	resources/views/	HTML con sintaxis Blade (@if, @foreach, {{ }})
Controlador	app/Http/Controllers/	Recibe petición, consulta BD, devuelve vista
Rutas	routes/web.php	Une URL + método HTTP con el controlador

2. Base de datos y relaciones

Diagrama de tablas



Relaciones clave

- Un **Offer** tiene muchos **ProductOffer** (los platos incluidos en esa oferta)
- Un **Product** puede estar en muchos **ProductOffer** (el mismo plato en varias ofertas)
- Un **ProductOffer** puede tener un precio propio distinto al del Product base
- Un **Order** pertenece a un **User** y tiene muchos **ProductOrder**
- Un **ProductOrder** apunta a un **ProductOffer** concreto (no directamente al Product)

Configuración PostgreSQL en **.env**

```

DB_CONNECTION=pgsql
DB_HOST=127.0.0.1
DB_PORT=5432
DB_DATABASE=prieto_eats
DB_USERNAME=admin
DB_PASSWORD=admin123
SESSION_DRIVER=database
  
```

Por qué SESSION_DRIVER=database: las sesiones (y el carrito) se guardan en la tabla **sessions** de PostgreSQL, que se creó con la migración base de Laravel.

3. Modelos Eloquent

Offer — `app/Models/Offer.php`

```
protected $fillable = ['date_delivery', 'time_delivery', 'datetime_limit'];

protected $casts = [
    'date_delivery' => 'date',    // convierte el string en objeto Carbon
    automáticamente
];

public function productsOffer()
{
    return $this->hasMany(ProductOffer::class); // una oferta tiene muchos
    ProductOffer
}
```

\$casts: hace que `$offer->date_delivery` sea ya un objeto Carbon sin necesidad de `Carbon::parse()`. Por eso en la vista podemos hacer directamente `$offer->date_delivery->format('d/m/Y')`.

Product — app/Models/Product.php

```
protected $fillable = ['name', 'description', 'price', 'image'];

public function productsOffer()
{
    return $this->hasMany(ProductOffer::class); // un producto puede estar en
    muchas ofertas
}
```

ProductOffer — tabla intermedia con datos propios

```
protected $fillable = ['offer_id', 'product_id', 'price'];

public function offer()    { return $this->belongsTo(Offer::class); }
public function product() { return $this->belongsTo(Product::class); }
public function productsOrder() { return $this->hasMany(ProductOrder::class); }
```

Esta tabla NO es una pivot simple porque tiene su propio **price** (el precio del plato dentro de esa oferta concreta puede ser diferente al precio base del producto). Por eso tiene modelo propio.

Order y ProductOrder

```
// Order.php
protected $fillable = ['user_id', 'total'];

public function user()    { return $this->belongsTo(User::class); }
public function products() { return $this->hasMany(ProductOrder::class); }
```

```
'order_id'); }

// ProductOrder.php
protected $fillable = ['order_id', 'product_offer_id', 'quantity'];

public function order() { return $this->belongsTo(Order::class); }
public function productOffer() { return $this->belongsTo(ProductOffer::class); }
```

User — con campo `is_admin`

```
protected $fillable = ['name', 'email', 'password', 'is_admin'];

public function isAdmin()
{
    return $this->is_admin; // true o false (booleano)
}
```

El campo `is_admin` se añadió con una migración independiente posterior:

```
Schema::table('users', function (Blueprint $table) {
    $table->boolean('is_admin')->default(false);
});
```

4. Sistema de rutas

Archivo `routes/web.php` — estructura completa

```
/          → redirect a /prieto          (pública)
/prieto     → prietoController@mostrar    (pública – home con ofertas)
/login_prieto GET → mostrar formulario    (pública)
/login_prieto POST → procesar login       (pública)
/register_prieto GET → mostrar formulario (pública)
/register_prieto POST → crear usuario      (pública)

— middleware('auth') —————
/logout_prieto POST → cerrar sesión
/cartShow      GET → ver carrito
/cartAdd/{id}  POST → añadir producto al carrito
/cartRemove/{id} DELETE → quitar producto
/cartClear     DELETE → vaciar carrito
/cartAddOne/{id} POST → +1 cantidad
/cartRemoveOne/{id} POST → -1 cantidad
/cartOrder     POST → confirmar pedido
/ordersShow    GET → ver mis pedidos

— middleware(['auth','isAdmin']) + prefix('admin') —————
```

```

/admin/products      → resource CRUD productos
/admin/offers        → resource CRUD ofertas

```

Route::resource — qué genera automáticamente

```
Route::resource("products", productController::class);
```

Esto crea **7 rutas** automáticamente:

Verbo	URL	Nombre	Método
GET	/admin/products	admin.products.index	index()
GET	/admin/products/create	admin.products.create	create()
POST	/admin/products	admin.products.store	store()
GET	/admin/products/{product}	admin.products.show	show()
GET	/admin/products/{product}/edit	admin.products.edit	edit()
PUT/PATCH	/admin/products/{product}	admin.products.update	update()
DELETE	/admin/products/{product}	admin.products.destroy	destroy()

Grupos de middleware en rutas

```

// Solo usuarios logueados
Route::middleware("auth")->group(function () {
    Route::get("/cartShow", [cartController::class, 'cartShow'])-
>name("cartShow");
    // ...
});

// Solo administradores (auth + isAdmin)
Route::middleware(["auth", "isAdmin"])
->prefix("admin")           // todas las URLs empiezan por /admin/
->name("admin.")           // todos los nombres empiezan por admin.
->group(function () {
    Route::resource("products", productController::class);
});

```

prefix('admin'): añade /admin/ delante de todas las URLs del grupo.

name('admin.'): añade admin. delante de todos los nombres de ruta, por eso en las vistas usamos route('admin.products.index').

Cómo usar rutas en las vistas

```

{{-- URL de una ruta nombrada --}}
<a href="{{ route('home_prieto') }}">Inicio</a>

{{-- URL con parámetro --}}
<a href="{{ route('admin.products.edit', $product) }}">Editar</a>

{{-- Formulario POST con CSRF --}}
<form method="POST" action="{{ route('cartAdd', $po->id) }}">
    @csrf
    <button type="submit">Añadir al carrito</button>
</form>

{{-- Formulario DELETE (HTML solo acepta GET/POST, se simula con @method) --}}
<form method="POST" action="{{ route('cartRemove', $po->id) }}">
    @csrf
    @method('DELETE')
    <button>Quitar</button>
</form>

```

@csrf: genera un token oculto que Laravel verifica para evitar ataques CSRF (Cross-Site Request Forgery). Sin él, Laravel devuelve error 419.

@method('DELETE'); los formularios HTML solo soportan GET y POST. Laravel interpreta el campo `_method` para simular PUT/PATCH/DELETE.

5. Autenticación: login, registro y logout

Login — `prietoController@store`

```

public function store(LoginRequest $request)
{
    $request->authenticate();           // valida credenciales contra la BD
    $request->session()->regenerate(); // genera nuevo ID de sesión (seguridad)
    return redirect()->intended('/prieto'); // redirige a donde quería ir, o a
    /prieto
}

```

LoginRequest: clase de Breeze que encapsula la validación del login (email requerido, password requerido) y llama a `Auth::attempt()` internamente.

regenerate(): cambia el ID de sesión para prevenir el ataque *Session Fixation* (alguien que conocía el ID de sesión anterior no puede aprovecharse).

redirect()->intended('/prieto'); si el usuario intentó acceder a una ruta protegida y fue redirigido al login, aquí lo manda de vuelta. Si no, va a `/prieto`.

Registro — `prietoController@storeRegister`

```

public function storeRegister(Request $request)
{
    $credentials = $request->validate([
        'name'      => 'required|string|max:255',
        'email'     => 'required|email|unique:users,email',
        'password'  => 'required|min:8|confirmed',
    ]);

    $user = User::create([
        'name'      => $credentials['name'],
        'email'     => $credentials['email'],
        'password'  => bcrypt($credentials['password']),
        'is_admin' => false,
    ]);

    Auth::login($user);           // logueamos al usuario recién creado
    return redirect('/prieto');
}

```

unique:users,email: la regla de validación comprueba que ese email no existe ya en la tabla **users**.

confirmed: exige que exista un campo **password_confirmation** con el mismo valor (el "repite contraseña").

bcrypt(): cifra la contraseña. Nunca se guarda en texto plano.

Auth::login(\$user): autentica al usuario sin necesidad de que introduzca credenciales de nuevo.

Logout — `prietoController@destroy`

```

public function destroy(Request $request)
{
    Auth::guard('web')->logout();           // cierra la sesión de autenticación

    $request->session()->invalidate();       // destruye todos los datos de sesión
    $request->session()->regenerateToken();  // regenera el token CSRF

    return redirect('/prieto');
}

```

Verificar autenticación en Blade

```

@auth
    {{-- Solo lo ven usuarios logueados --}}
    <p>Hola, {{ Auth::user()->name }}</p>
@else
    {{-- Solo lo ven visitantes --}}
    <p class="text-danger">Inicia sesión para comprar</p>
@endauth

```

```

{{-- También se puede así --}}
@if(Auth::check())
    <a href="{{ route('cartShow') }}">Carrito</a>
@endif

```

6. Vistas Blade: layout y parciales

Estructura de archivos de vista

```

resources/views/
├── layouts/
│   ├── plantilla.blade.php ← layout principal (Bootstrap 5)
│   ├── navbar.blade.php   ← barra de navegación (incluida en plantilla)
│   └── footer.blade.php    ← pie de página
├── auth/
│   ├── login.blade.php
│   └── register.blade.php
├── home.blade.php
├── cart/
│   └── show.blade.php
├── orders/
│   └── index.blade.php
├── admin/
│   ├── products/
│   │   ├── index.blade.php
│   │   ├── create.blade.php
│   │   └── edit.blade.php
│   └── offers/
│       ├── index.blade.php
│       └── create.blade.php

```

layouts/plantilla.blade.php — el layout principal

```

<!DOCTYPE html>
<html lang="es">
<head>
    <link href="bootstrap@5.3.0/css/bootstrap.min.css" rel="stylesheet">
    <style>
        .text-prieto { color: #2eab4f !important; }
        .btn-prieto { background-color: #2eab4f; color: white; }
    </style>
</head>
<body class="d-flex flex-column min-vh-100">

    @include('layouts.navbar')    {{-- incluye el navbar --}}

    <main class="flex-grow-1">
        @yield('content')        {{-- aquí se inyecta el contenido de cada vista -

```



```

-}}
</main>

<footer>...</footer>
<script src="bootstrap@5.3.0/js/bootstrap.bundle.min.js"></script>
</body>
</html>

```

Cómo usa el layout cada vista

```

{{-- home.blade.php --}}
@extends('layouts.plantilla')    {{-- hereda el layout --}}

@section('content')              {{-- rellena el @yield('content') --}}
<div class="container mt-4">
    <h2>Nuestras ofertas</h2>
    ...
</div>
@endsection

```

@extends: indica de qué layout hereda la vista.

@yield('content'); marca el hueco donde se insertará el contenido.

@section('content') ... @endsection: define el contenido que va en ese hueco.

@include('layouts.navbar'); incluye otro archivo Blade, como un **include** de PHP.

Navbar — lógica de autenticación

```

@auth
    {{-- Si está logueado: carrito + dropdown usuario --}}
    <a href="{{ route('cartShow') }}">Carrito</a>

    <div class="dropdown">
        {{ Auth::user()->name }}
        <div class="dropdown-menu">
            <a href="{{ route('ordersShow') }}">Mis pedidos</a>

            @if(Auth::user()->is_admin)
                {{-- Solo visible para admins --}}
                <a href="{{ route('admin.products.index') }}">Productos</a>
                <a href="{{ route('admin.offers.index') }}">Ofertas</a>
            @endif

            <form method="POST" action="{{ route('logout_prieto') }}">
                @csrf
                <button>Cerrar sesión</button>
            </form>
        </div>
    </div>
@else

```

```

    {{-- Si NO está logueado: login y registro --}}
    <a href="{{ route('login_prieto') }}">Login</a>
    <a href="{{ route('register_prieto') }}">Registrarse</a>
@endauth

```

7. Home: ofertas con tabs y cards

Controlador — `prietoController@mostrar`

```

public function mostrar()
{
    $offers = Offer::with("productsOffer.product") // Eager Loading
        ->where(function ($q) {
            $q->whereNull("datetime_limit") // sin límite → siempre
visible
            ->orWhere("datetime_limit", ">", now()); // o límite aún no pasado
        })
        ->orderBy("date_delivery", "asc")
        ->orderBy("time_delivery", "asc")
        ->get();

    return view("home", compact("offers"));
}

```

Puntos importantes:

Eager Loading con `with("productsOffer.product")`:

Laravel carga de una vez, con 3 consultas en total, toda la información:

```

SELECT * FROM offers WHERE ...
SELECT * FROM product_offers WHERE offer_id IN (1, 2, 3)
SELECT * FROM products WHERE id IN (1, 2, 3, 4, 5)

```

Sin Eager Loading se harían N+1 consultas (una por cada oferta y producto).

Filtro de ofertas activas:

```

->where(function ($q) {
    $q->whereNull("datetime_limit") // null = sin caducidad
    ->orWhere("datetime_limit", ">", now()); // o todavía en plazo
})

```

`compact("offers")`: crea el array `['offers' => $offers]` para pasar a la vista.

Vista — tabs Bootstrap + cards de producto

```

{{-- Una pestaña por oferta --}}
<ul class="nav nav-tabs">
    @foreach($offers as $index => $offer)
        <li class="nav-item">
            <button class="nav-link {{ $index === 0 ? 'active' : '' }}"
                data-bs-toggle="tab"
                data-bs-target="#offer-{{ $offer->id }}"
                {{-- date_delivery ya es Carbon por el cast del modelo --}}
                {{ $offer->date_delivery->locale('es')->isoFormat('D [de] MMMM') }}
            </button>
        </li>
    @endforeach
</ul>

{{-- Contenido de cada tab --}}
@foreach($offers as $index => $offer)
    <div class="tab-pane fade {{ $index === 0 ? 'show active' : '' }}"
        id="offer-{{ $offer->id }}">

        @foreach($offer->productsOffer as $po)
            @php
                $p = $po->product;
                $precio = $po->price ?? $p->price ?? 0; // precio oferta o precio
base
            @endphp

            <div class="col-md-4">
                <div class="card">
                    
                    <div class="card-body">
                        <h5>{{ $p->name }}</h5>
                        <p>Precio: {{ number_format($precio, 2) }} €</p>

                        @auth
                            <form action="{{ route('cartAdd', $po->id) }}"
method="POST">
                                @csrf
                                <button class="btn btn-success btn-sm">Añadir al
carrito</button>
                            </form>
                        @else
                            <p class="text-danger">Inicia sesión para comprar</p>
                        @endauth
                    </div>
                </div>
            </div>
        @endforeach
    </div>
@endforeach

```

`$po->price ?? $p->price ?? 0`: operador null-coalescing. Si el ProductOffer tiene precio propio, lo usa. Si no, usa el precio base del producto. Si tampoco tiene, 0.

`asset($p->image)`: genera la URL completa del asset, por ejemplo
`http://localhost/storage/img/pollo.png`.

`$offer->date_delivery->locale('es')->isoFormat('D [de] MMMM')`: el cast convierte la fecha a Carbon, `locale('es')` activa el español, `isoFormat` da el formato "14 de mayo".

8. Carrito de la compra con sesiones

Estructura del carrito en sesión

```
// Cómo está guardado en la sesión (clave "cart"):
$cart = [
    "offer_id" => 2,           // ID de la oferta activa
    "items" => [
        5 => [                // clave = product_offer_id
            "product_offer_id" => 5,
            "qty" => 2,
        ],
        7 => [
            "product_offer_id" => 7,
            "qty" => 1,
        ],
    ],
];
```

Solo se puede tener en el carrito productos de **una sola oferta**. Si intentas mezclar, el carrito se vacía automáticamente con un mensaje flash.

Métodos privados helper

```
private function getCart(Request $request): array
{
    return $request->session()->get("cart", [
        "offer_id" => null,
        "items" => []
    ]);
}

private function saveCart(Request $request, array $cart): void
{
    $request->session()->put("cart", $cart);
}
```

Se usan estos helpers para no repetir `session()->get("cart")` en cada método.

cartAdd — añadir producto

```

public function cartAdd(Request $request, $id)
{
    $po = ProductOffer::with("offer")->findOrFail($id);
    $cart = $this->getCart($request);

    // Prevenir mezcla de ofertas distintas
    if ($cart["offer_id"] !== null && (int)$cart["offer_id"] !== (int)$po->offer_id) {
        $cart = ["offer_id" => null, "items" => []]; // resetear carrito
        $request->session()->flash("info", "Carrito reiniciado.");
    }

    $cart["offer_id"] = $po->offer_id;

    if (!isset($cart["items"][$po->id])) {
        $cart["items"][$po->id] = ["product_offer_id" => $po->id, "qty" => 1];
    } else {
        $cart["items"][$po->id]["qty"]++; // ya existe: incrementar cantidad
    }

    $this->saveCart($request, $cart);
    return redirect()->back(); // vuelve a la página anterior
}

```

cartRemoveOne — decrementar cantidad

```

public function cartRemoveOne(Request $request, $id)
{
    $cart = $this->getCart($request);

    if (isset($cart["items"][$id])) {
        $cart["items"][$id]["qty"]--;

        if ($cart["items"][$id]["qty"] <= 0) {
            unset($cart["items"][$id]); // si llega a 0, se elimina del carrito
        }
    }

    if (empty($cart["items"])) {
        $cart["offer_id"] = null; // si el carrito queda vacío, resetear offer_id
    }

    $this->saveCart($request, $cart);
    return redirect()->route("cartShow");
}

```

unset(\$array[\$key]): elimina un elemento de un array en PHP.

cartShow — mostrar carrito

```

public function cartShow(Request $request)
{
    $cart = $this->getCart($request);
    $items = $cart["items"];
    $productOfferIds = array_keys($items); // [5, 7, ...]

    // Carga de BD con Eager Loading en UNA sola consulta
    $productOffers = ProductOffer::with(["product", "offer"])
        ->whereIn("id", $productOfferIds)
        ->get()
        ->keyBy("id"); // convierte la colección en array clave=>objeto

    $lineas = [];
    $total = 0;

    foreach ($items as $poId => $row) {
        if (!isset($productOffers[$poId])) continue;

        $po = $productOffers[$poId];
        $qty = (int)$row["qty"];
        $precio = (float)($po->price ?? $po->product->price ?? 0);
        $subtotal = $qty * $precio;
        $total += $subtotal;

        $lineas[] = ["po" => $po, "qty" => $qty, "precio" => $precio, "subtotal"
=> $subtotal];
    }

    return view("cart.show", compact("lineas", "total", "offer"));
}

```

->keyBy("id"): convierte [objeto, objeto, ...] en [id => objeto, id => objeto] para acceder por clave directamente con \$productOffers[\$poId].

array_keys(\$items): devuelve solo las claves del array (los IDs de ProductOffer).

Mensajes flash en sesión

```

// En el controlador:
$request->session()->flash("info", "Carrito reiniciado.");

// En la vista:
@if(session('info'))
    <div class="alert alert-info">{{ session('info') }}</div>
@endif

```

Los mensajes flash solo duran **una petición**. Se guardan en sesión pero se eliminan automáticamente después de leerlos.

9. Confirmar pedido (transacción)

```
public function cartOrder(Request $request)
{
    $cart = $this->getCart($request);

    if (empty($cart["items"])) {
        return redirect()->route("cartShow")->withErrors(["error" => "El carrito
está vacío."]);
    }

    // Calcular total y preparar filas
    $total = 0;
    $rows = [];
    foreach ($items as $poId => $row) {
        $po = $productOffers[$poId];
        $qty = (int)($row["qty"] ?? 1);
        $price = (float)($po->price ?? $po->product->price ?? 0);
        $total += $qty * $price;

        $rows[] = ["product_offer_id" => $po->id, "quantity" => $qty];
    }

    $orderId = null;

    // Transacción: las dos operaciones ocurren juntas o ninguna
    DB::transaction(function () use ($total, $rows, &$orderId) {
        $order = Order::create([
            "user_id" => auth()->id(),
            "total" => $total,
        ]);

        $orderId = $order->id;

        $order->products()->createMany($rows); // inserta todas las líneas a la
vez
    });

    $request->session()->forget("cart"); // vaciar el carrito tras el pedido

    return redirect()->route("ordersShow")->with("info", "Pedido #$orderId
realizado.");
}
```

Por qué una transacción

```
DB::transaction(function() {
    // Si Order::create falla → no se crean ProductOrders
    // Si createMany falla → se deshace también el Order
```

```
// Solo si TODO va bien    → se confirma (commit)
});
```

Sin transacción podría crearse el **Order** pero fallar al crear los **ProductOrder**, dejando la BD en un estado inconsistente.

createMany(\$rows): inserta un array de filas de una vez, más eficiente que hacer un **create()** por cada fila.

10. Roles y middleware IsAdmin

Cómo funciona el middleware

app/Http/Middleware/IsAdmin.php:

```
public function handle(Request $request, Closure $next): Response
{
    if (Auth::check() && Auth::user()->is_admin) {
        return $next($request); // deja pasar la petición
    }

    abort(403, "No eres Administrador!!"); // para la petición con error 403
}
```

Registro en **bootstrap/app.php**:

```
->withMiddleware(function (Middleware $middleware): void {
    $middleware->alias([
        "isAdmin" => \App\Http\Middleware\IsAdmin::class,
    ]);
});
```

Uso en rutas:

```
Route::middleware(["auth", "isAdmin"])
    ->prefix("admin")
    ->name("admin.")
    ->group(function () {
        Route::resource("products", productController::class);
        Route::resource("offers", offerController::class);
    });
```

El middleware **auth** se ejecuta primero. Si el usuario no está logueado, lo manda al login antes de llegar a comprobar **isAdmin**.

Cómo se protege también en las vistas

```
{{-- Forma 1: comprobar campo directo --}}
@if(Auth::user()->is_admin)
    <a href="{{ route('admin.products.index') }}">Panel Admin</a>
@endif

{{-- Forma 2: usar el método del modelo --}}
@if(Auth::user()->isAdmin())
    <a href="{{ route('admin.offers.index') }}">Ofertas</a>
@endif
```

IMPORTANTE: la protección en Blade es solo visual. La protección real está en el middleware de las rutas. Si alguien accede directamente a `/admin/products` sin ser admin, el middleware devuelve 403.

Cómo hacer un usuario admin (Tinker)

```
php artisan tinker
>>> $user = App\Models\User::find(2);
>>> $user->is_admin = true;
>>> $user->save();
```

O con el **UserSeeder** que ya está configurado:

- Email: `admin@mail.com`
- Password: `12345678`
- is_admin: `1` (true)

11. CRUD de Productos (Admin)

Crear producto con imagen — `productController@store`

```
public function store(Request $request)
{
    $data = $request->validate([
        "name"          => "required|string|max:255",
        "description"    => "nullable|string",
        "price"          => "required|numeric|min:0",
        "image"          => "nullable|image|mimes:jpg,jpeg,png,webp|max:2048",
    ]);

    $product = new Product();
    $product->name          = $data["name"];
    $product->description   = $data["description"] ?? null;
    $product->price         = $data["price"];
```

```

if ($request->hasFile("image")) {
    // Guarda en storage/app/public/img/ y devuelve "img/nombreadarchivo.png"
    $path = $request->file("image")->store("img", "public");
    $product->image = "storage/" . $path; // guardamos "storage/img/xxx.png"
}

$product->save();
return redirect()->route("admin.products.index");
}

```

store("img", "public"): guarda el archivo en `storage/app/public/img/`. Para que sea accesible desde el navegador se necesita ejecutar `php artisan storage:link` que crea un enlace simbólico de `public/storage` → `storage/app/public`.

asset(\$product->image) en la vista genera: `http://localhost/storage/img/archivo.png`.

Formulario con **enctype** para subir archivos

```

<form action="{{ route('admin.products.store') }}" method="POST"
enctype="multipart/form-data">
    @csrf
    <input type="text" name="name" required>
    <input type="number" step="0.01" name="price" required>
    <input type="file" name="image" accept="image/*">
    <button type="submit">Guardar</button>
</form>

```

enctype="multipart/form-data": imprescindible para formularios que suben archivos. Sin él, `$request->file("image")` devuelve null.

Mostrar errores de validación en la vista

```

<input type="text" name="name"
class="form-control @error('name') is-invalid @enderror"
value="{{ old('name') }}">

@error('name')
    <div class="invalid-feedback">{{ $message }}</div>
@enderror

```

@error('name'): comprueba si hay error de validación para ese campo.

old('name'): recupera el valor que introdujo el usuario antes de que fallara la validación, para no perderlo.

is-invalid: clase Bootstrap que pone el borde rojo en el input.

12. Crear Ofertas con checkboxes (Admin)

Vista — `admin/offers/create.blade.php`

```

<form action="{{ route('admin.offers.store') }}" method="POST">
    @csrf

    <input type="date" name="date_delivery" required>
    <input type="time" name="time_delivery" required>
    <input type="datetime-local" name="datetime_limit">

    <table>
        @foreach($productos as $p)
            <tr>
                <td>
                    <input type="checkbox" name="dish_selected[]" value="{{ $p->id
}}">

                </td>
                <td>{{ $p->name }}</td>
                <td>{{ $p->price }} €</td>
                <td>
                    <input type="number" step="0.01" name="dish_price[{{ $p->id }}]">
                </td>
            </tr>
        @endforeach
    </table>

    <button>Guardar oferta</button>
</form>

```

`name="dish_selected[]"`: el `[]` hace que PHP reciba un array. Si marcas los productos 2 y 5, llega `dish_selected = [2, 5]`.

`name="dish_price[{{ $p->id }}]"`: crea un array asociativo. Si el producto 2 tiene precio 7.50, llega `dish_price = [2 => 7.50, 5 => null]`.

Controlador — `offerController@store`

```

public function store(Request $request)
{
    $validated = $request->validate([
        "date_delivery" => "required|date|after:today",    // debe ser fecha
futura
        "time_delivery" => "required",
        "datetime_limit" => "nullable|date|after:now",      // opcional, pero si
viene debe ser futuro
        "dish_selected" => "required|array|min:1",          // al menos un
producto
        "dish_selected.*" => "integer|distinct|exists:products,id", // IDs
válidos, sin repetir
        "dish_price" => "nullable|array",
        "dish_price.*" => "nullable|numeric|min:0",
    ]);
}

```

```

DB::transaction(function () use ($validated) {

    // 1. Crear la oferta
    $oferta = Offer::create([
        "date_delivery" => $validated["date_delivery"],
        "time_delivery" => $validated["time_delivery"],
        "datetime_limit" => $validated["datetime_limit"] ?? null,
    ]);

    // 2. Preparar las filas de ProductOffer
    $products = Product::whereIn("id", $validated["dish_selected"])
        ->get()->keyBy("id");

    $rows = [];
    foreach ($validated["dish_selected"] as $productId) {
        $precioFormulario = $validated["dish_price"][$productId] ?? null;
        $precioFinal = ($precioFormulario !== null)
            ? $precioFormulario
            : $products[$productId]->price; // usa precio base si no pusieron
        uno

        $rows[] = ["product_id" => $productId, "price" => $precioFinal];
    }

    // 3. Insertar todas las líneas de ProductOffer
    $oferta->productsOffer()->createMany($rows);
});

return redirect()->route('admin.offers.index');
}

```

Flujo completo al crear una oferta:

1. Admin marca checkboxes de productos y rellena el formulario
2. Se envía POST /admin/offers
3. El controlador valida:
 - date_delivery: fecha válida y posterior a hoy
 - dish_selected: array de IDs de productos que existen en BD
4. Dentro de una transacción:
 - a) Se crea el registro en tabla offers
 - b) Para cada producto seleccionado se crea un ProductOffer con su precio (formulario o precio base del producto)
5. Redirige al listado de ofertas

after:today: la fecha debe ser estrictamente posterior a hoy.

distinct: no se puede seleccionar el mismo producto dos veces.

exists:products,id: verifica que el ID existe en la tabla products.

13. Seeders y datos de prueba

Orden de ejecución en `DatabaseSeeder.php`

```
$this->call([
    ProductsSeeder::class,      // 1. Productos primero (sin dependencias)
    OffersSeeder::class,        // 2. Ofertas (sin dependencias)
    OfferProductSeeder::class,  // 3. ProductOffers (depende de products y offers)
    UserSeeder::class,          // 4. Admin (independiente)
]);
```

El orden importa por las claves foráneas. Si ejecutas `OfferProductSeeder` antes que `OffersSeeder`, falla porque intenta referenciar un `offer_id` que no existe.

`OffersSeeder` — fechas dinámicas

```
$tomorrow = now()->addDay();
$dayAfter  = now()->addDays(2);
$threeDays = now()->addDays(3);

DB::table('offers')->insert([
    'date_delivery' => $tomorrow->format('Y-m-d'),
    'time_delivery' => '13:30',
    'datetime_limit' => now()->addDay()->format('Y-m-d H:i:s'),
]);
```

Se usa `now()->addDay()` en lugar de una fecha fija para que las ofertas sean **siempre actuales** cuando se ejecute el seeder. Con fechas fijas como `'2025-01-27'`, las ofertas caducarían pasada esa fecha y la home aparecería vacía.

Comandos útiles

```
# Crear tablas y ejecutar todos los seeders desde cero
php artisan migrate:fresh --seed

# Solo ejecutar los seeders (sin borrar tablas)
php artisan db:seed

# Ejecutar un seeder concreto
php artisan db:seed --class=OffersSeeder

# Crear enlace simbólico para las imágenes
php artisan storage:link

# Consola interactiva de Laravel (para probar Eloquent)
php artisan tinker
```

14. Conceptos clave para defender el proyecto

¿Qué es Eloquent y por qué usarlo?

Eloquent es el ORM (Object-Relational Mapping) de Laravel. Permite trabajar con la BD usando objetos PHP en lugar de SQL puro.

```
// Con Eloquent:
$offers = Offer::with('productsOffer.product')->where('datetime_limit', '>',
now())->get();

// Equivalente SQL:
// SELECT * FROM offers WHERE datetime_limit > NOW()
// SELECT * FROM product_offers WHERE offer_id IN (...)
// SELECT * FROM products WHERE id IN (...)
```

¿Qué es Eager Loading y por qué evita el problema N+1?

```
// MAL - problema N+1: 1 consulta offers + N consultas (una por oferta)
$offers = Offer::all();
foreach ($offers as $offer) {
    echo $offer->productsOffer; // nueva consulta por cada oferta
}

// BIEN - Eager Loading: siempre 3 consultas, independientemente del número de
ofertas
$offers = Offer::with('productsOffer.product')->get();
```

¿Qué es una transacción de BD y cuándo se usa?

Una transacción agrupa varias operaciones de BD en un bloque atómico: o se ejecutan todas o ninguna.

```
DB::transaction(function () {
    $order = Order::create([...]); // si falla aquí...
    $order->products()->createMany($rows); // ...esto no se ejecuta
});
```

Se usa en `cartOrder()` porque crear el `Order` y sus `ProductOrder` deben ocurrir juntos. Si solo se crea el `Order` pero fallan las líneas, tenemos un pedido sin productos en BD.

¿Cómo funciona el sistema de sesiones?

1. Usuario visita la app → Laravel genera un ID de sesión único
2. Ese ID se guarda en una cookie del navegador (`laravel_session`)

3. En cada petición, el navegador envía esa cookie
4. Laravel busca la sesión en la BD (tabla sessions) por ese ID
5. Carga los datos de sesión (incluido el carrito)
6. Al hacer logout: se destruye la sesión y se genera un nuevo ID

¿Por qué el carrito solo permite una oferta a la vez?

Porque cada oferta tiene su propia fecha y hora de recogida. Si un usuario mezcla productos de la oferta del martes y del jueves, sería imposible gestionar la entrega. Por eso en `cartAdd()`:

```
if ($cart["offer_id"] !== null && (int)$cart["offer_id"] !== (int)$po->offer_id) {
    $cart = ["offer_id" => null, "items" => []]; // resetear si hay cambio de
    oferta
}
```

¿Qué hace `Route::resource` exactamente?

Genera las 7 rutas estándar CRUD automáticamente. En el proyecto:

```
Route::resource("products", productController::class);
// Genera: admin.products.index, admin.products.create, admin.products.store,
//         admin.products.show, admin.products.edit, admin.products.update,
//         admin.products.destroy
```

¿Por qué el precio está en `ProductOffer` y no solo en `Product`?

El mismo plato (Product) puede aparecer en distintas ofertas a precios diferentes. Por ejemplo, el "Pollo asado" puede costar 12€ en la oferta del martes y 10€ en la del jueves. `ProductOffer.price` permite este precio específico por oferta, mientras que `Product.price` es el precio base de referencia.

¿Qué es `@csrf` y para qué sirve?

CSRF son las siglas de *Cross-Site Request Forgery*. Es un ataque donde una web maliciosa hace que el navegador del usuario envíe una petición a tu app sin que él lo sepa.

`@csrf` genera un campo oculto `<input type="hidden" name="_token" value="...">`. Laravel verifica ese token en cada petición POST/PUT/DELETE. Si no coincide, devuelve error 419.

¿Qué diferencia hay entre `session()->put()`, `session()->flash()` y `session()->forget()`?

Método	Comportamiento
<code>session()->put('clave', \$valor)</code>	Guarda permanentemente hasta que se borre
<code>session()->flash('clave', \$valor)</code>	Solo dura una petición, luego desaparece automáticamente

Método	Comportamiento
<code>session()->forget('clave')</code>	Elimina la clave de la sesión
<code>session()->flush()</code>	Elimina toda la sesión

En el proyecto: el carrito usa `put()` (persiste entre páginas). Los mensajes de éxito/info usan `flash()` (se muestran una sola vez).

*Proyecto realizado por alumno/a de 2º Desarrollo de Aplicaciones Web
IES Gregorio Prieto — Programación Web en entorno servidor — Laravel*